Deterministic Measurement of Ontology-Grounded Hallucination and Solvability Detection in LLMs Using Twisty Puzzles

Vlad Flaks
OWOX
fv@owox.com

Mykhailo Flaks
Helmholtz Institute Mainz, Johannes Gutenberg University Mainz
mflaks@uni-mainz.de

[**TODO:** date of submission] arXiv:[**VALUE:** ID] [cs.CL]

Abstract

Many evaluations treat a wrong move sequence from a Large Language Model (LLM) as a search error. This view hides a second type of error. A model can use a move that does not exist, or it can try to solve a state that no legal move sequence reaches.

We ask whether the rules of the puzzle in the prompt change how often each error occurs. Twisty puzzles make this question measurable, because the puzzle defines every move and a deterministic validator checks every answer. No person and no second model judges correctness.

Nine Artificial Intelligence (AI) models receive each frozen 2x2 or 3x3 cube state under three setups: no rules, prose rules, or an exact move list. A corruption step turns one third of the inputs into unreachable states, and the correct answer to them is a mismatch flag. We report success, hallucination, and mismatch detection rates with 95% Wilson intervals, plus efficiency, consistency, tokens, and cost per success. With the exact move list, the hallucination rate changes from [**VALUE:** rate, no rules] to [**VALUE:** rate, move list], and mismatch detection reaches [**VALUE:** rate].

1 Introduction

A common view holds that a Large Language Model (LLM) fails a puzzle because it does not search deep enough. Under this view, a larger reasoning budget or a better search strategy repairs the error. Benchmarks that score the distance to the solved state support this view, because they count each failure as a short plan [1, 2]. A score of this type cannot show whether the model knew which moves exist.

Take a solved 3x3 cube and twist one corner in place. The cube now looks one move away from solved, but no sequence of legal face turns can solve it. A model that returns a move sequence for this state has not made a search error. It has missed a rule of the domain, and more search cannot repair that.

This paper asks a narrow question. We keep the Artificial Intelligence (AI) model and the puzzle state constant and change only the rules in the prompt. Does the rate of impossible moves change? We also ask whether the model notices when a state is unreachable. A script can check both answers, so we can measure them without a judge model.

We answer with a closed domain in which the rules are exact. A formal ontology defines the pieces, the legal move group, and the reachability axioms of each cube. A deterministic validator then scores every attempt from the raw response alone. Figure 1 shows the four workflows and the separation between valid and corrupted inputs.

This work makes three contributions. The list below states each contribution and the part of the paper that supports it. [**TODO:** Replace the third item with the measured main effect when Release 1.1.2 results exist.]

- We release a frozen benchmark of 90 cube states with a recorded seed. One third of the states are unreachable by construction, so the correct answer to them is a flag and not a move list.
- We define hallucination as one countable event: a move string that is not in the legal move group. A deterministic validator gives each attempt one of five outcomes, and no model judges another model.
- We compare nine AI models from three providers under three prompt setups on byte-identical inputs. [**TODO:** One sentence with the main measured effect and its 95% interval.]

2 Related Work

Several benchmarks use cubes and other puzzles with moves that a program can check. Cube Bench tests multimodal models on 3x3 cube skills and scores progress as the distance to solved [1]. CubeBench uses the cube to test agents that track state over many steps under partial observation [3]. Other studies find invalid moves on the 8-puzzle and a collapse of reasoning models past a complexity limit [2, 4, 5].

The closest match to our definition of hallucination lists illegal notation as one failure mode on finite puzzles [6].

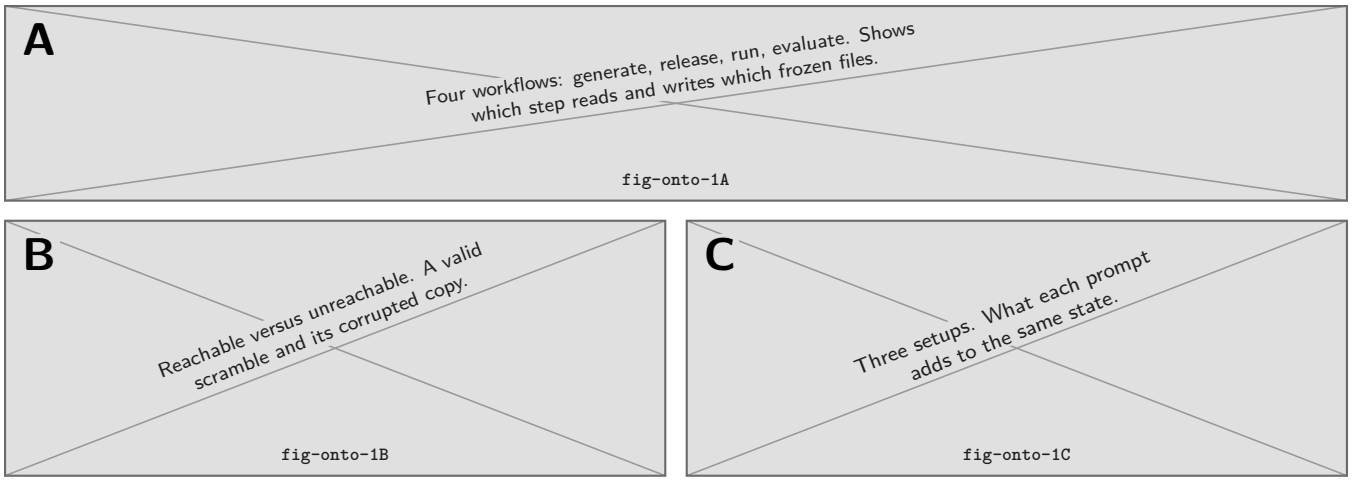


Figure 1: Benchmark workflow and design. (A) The four workflows `generate`, `release`, `run`, and `evaluate`, with the frozen release folder as the only interface between them. (B) A valid scramble and its corrupted copy with one twisted corner, which breaks the corner orientation invariant. (C) The content that each setup adds to the same rendered state.

Other studies put explicit rules into the prompt and check with code whether models keep them [7–9]. HalluLens gives a broad taxonomy of hallucination [10]. We narrow hallucination to one countable event, an impossible move.

A separate line of work asks whether models notice a question without a valid answer [11–14]. Work on ontologies and knowledge graphs adds domain structure to the prompt or checks answers against it [15–18]. Most of these studies use a reference answer or a small set of examples, and not a deterministic rule check over frozen inputs.

None of these studies gives the same frozen inputs to several models under three levels of rules in the prompt. None measures success, impossible moves, and detection of invalid inputs with one deterministic validator. That combination is the gap that this study fills. [**TODO:** Add two or three 2026 papers on solvability detection if a literature pass finds them.]

3 Scope and Terms

An AI model that plans without a formal domain ontology cannot separate a hard search problem from an impossible state. In open-ended text tasks, people or secondary LLMs judge whether a response contains a hallucination. Judge models add variance and cannot show whether an error came from shallow planning or from missing rules. Twisty puzzles remove judge variance, because every state is discrete and every move is deterministic.

Table 1 lists the controlled vocabulary of this paper. Each term has one meaning in the code, the configuration files, the raw records, and this text. The table also gives the code location that defines the term.

4 Method

4.1 Algebraic ontology and turn operator

An LLM reads a puzzle state as a flat sequence of characters, but the physical puzzle obeys a closed algebraic ontology. We define the puzzle ontology as a five-tuple over finite discrete sets. Equation (1) states this structure.

$$\mathcal{O} = (\mathcal{C}, \mathcal{I}, \mathcal{R}, \mathcal{T}, \mathcal{A}) \tag{1}$$

In Equation (1), $\mathcal{C}$ denotes concepts, $\mathcal{I}$ individuals, $\mathcal{R}$ spatial relations, $\mathcal{T}$ the move group, and $\mathcal{A}$ the reachability axioms. Each sticker on an $n \times n$ cube has a position vector $\mathbf{p}$ and a unit normal $\mathbf{u}$. A clockwise quarter turn of face f rotates every sticker with $\mathbf{a} \cdot \mathbf{p} = n - 1$ around the face normal $\mathbf{a}$. The function `_turn_clockwise()` applies Equation (2) to each vector.

$$\mathrm{Rot}_{\mathbf{a}}(\mathbf{v}) = (\mathbf{a} \cdot \mathbf{v})\,\mathbf{a} - (\mathbf{a} \times \mathbf{v}) \tag{2}$$

4.2 Reachability invariants

This single operator generates all 18 face turns in `legal_moves()`. The 18 moves are the clockwise turns f, the anticlockwise turns $f' = f^{-1}$, and the half turns $f2 = f \circ f$. Every legal move preserves the three invariants in Equation (3).

$$\begin{aligned} &\textstyle\sum_{i=1}^{8} v_i \equiv 0 \pmod 3, \quad \sum_{j=1}^{12} w_j \equiv 0 \pmod 2, \\ &\operatorname{sgn}\sigma = \operatorname{sgn}\tau \end{aligned} \tag{3}$$

Here $v_i \in \{0, 1, 2\}$ is the orientation of corner i and $w_j \in \{0, 1\}$ is the flip of edge j. The last condition requires equal parity of the corner permutation σ and the edge permutation τ. Legal moves therefore reach 43,252,003,274,489,856,000 states, which is one twelfth of all assembled configurations.

Table 1: Controlled vocabulary. The second column gives the code symbol or file that defines each term in the repository [19].

Term	Code symbol or location	Definition
Ontology model	`prompts/{setup}/`	The rules of a puzzle: pieces, legal move group, state transitions, and reachability constraints.
Setup	`SETUPS` (`src/run/prompts.py`)	The prompt condition: `no_model`, `descriptive`, or `prescriptive`.
State	`state: str` (`src/puzzles/cube.py`)	A string of $6n^2$ face letters in `URFDLB` order, one letter for each sticker.
Legal move	`engine.legal_moves()`	One of the 18 Singmaster face turns, matched as an exact string without normalization.
Impossible move	`move not in legal` (`validator.py:65`)	Any returned string that is not in `legal_moves()`, including slice moves, lowercase turns, and invented tokens.
Valid or invalid input	`validity` (`inputs.jsonl`)	A frozen state that satisfies or violates the group invariants in `_is_reachable()`.
Corruption	`engine.corrupt()` (`cube.py:286`)	One twisted corner or one flipped edge, applied to a copy of a valid state.
Mismatch flag	`Verdict.mismatch`	The Boolean field `"mismatch"` in the JavaScript Object Notation (JSON) reply. It flags an unreachable state.
Hallucination	`Verdict.hallucination`	The outcome for any reply whose parsed moves contain at least one impossible move, even if the cube ends solved.
Reference length	`reference_length()`	The solve length of the reference solver in the Half-Turn Metric (HTM), for valid inputs only.

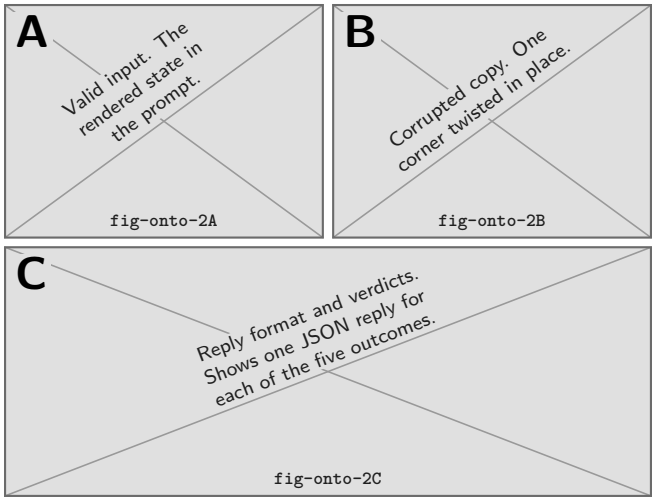


Figure 2: What the model receives and returns. (A) A valid input as the prompt shows it. (B) Its corrupted copy, which no legal move sequence reaches. (C) Example replies in the required JSON format and the outcome that the validator gives each one.

The 2x2 cube has only the corner permutation and the corner twist invariant. With the down-back-left (`DBL`) corner fixed, the 2x2 cube has $7! \cdot 3^7 = 3{,}674{,}160$ reachable states. The function `corrupt()` makes each invalid input from a copy of a valid state. It twists one corner or flips one edge, so the corner sum or the edge sum leaves zero. Figure 2 shows one valid input, its corrupted copy, and the reply format.

Table 2: Template size in bytes for each setup. Release 1.0.0 holds the placeholder templates of 18 October 2026. Release 1.1.2 holds the approved templates of 23 October 2026.

	Release 1.0.0		Release 1.1.2	
Setup	2x2	3x3	2x2	3x3
`no_model`	298	298	690	690
`descriptive`	428	428	1422	1489
`prescriptive`	365	365	1536	1603

4.3 Prompt setups and frozen releases

The benchmark freezes 90 inputs in the file `inputs.jsonl`, with the seed 20261017. Each puzzle has 10 valid scrambles and 5 corrupted copies at each scramble depth of 5, 10, and 20. Each input goes through three setups. The `no_model` setup gives the state and the reply format only. The `descriptive` setup adds prose rules, and the `prescriptive` setup also lists every legal move.

The prompt text changed after the freeze of Release 1.0.0. The approved templates in `prompts/` are 2.3 to 4.4 times longer than the frozen ones, as Table 2 and Figure 3 show. A prompt change needs a new minor release, so the measured results use Release 1.1.2. This release keeps the 90 inputs of Release 1.0.0 byte for byte and adds the approved templates. Its manifest records a 12-character version for each template.

4.4 Deterministic validation and metrics

The function `validate()` judges every attempt from the input, the raw response, and the stop reason. It gives one of five outcomes in the strict priority order of

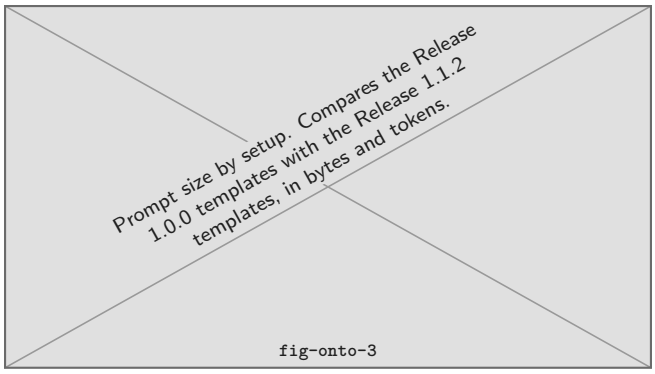


Figure 3: Prompt template size by setup and puzzle. Bars compare the Release 1.0.0 templates with the Release 1.1.2 templates. A second axis shows the mean rendered prompt length in input tokens under Release 1.1.2, [**VALUE:** tokens per setup].

Equation (4). A hallucination outcome holds even if the remaining moves solve the puzzle.

$$\texttt{infrastructure_failure} \succ \texttt{malformed_output} \succ \texttt{hallucination} \succ \texttt{success} \succ \texttt{failure} \tag{4}$$

A truncated or refused reply, or a reply without exactly the keys `moves` and `mismatch`, counts as malformed output. A valid input succeeds if all moves are legal, the cube ends solved, and the flag is off. An invalid input succeeds if all moves are legal and the flag is on. Any other attempt with only legal moves is a failure.

Every rate $p = k/n$ has a 95% Wilson score interval with $z = 1.96$ [20], as Equation (5) shows. Efficiency E is the mean ratio of move count to reference length over successful valid attempts. Consistency C is the mean share of the modal outcome over the repeated attempts on one input.

$$\mathrm{CI}_{95}(p, n) = \frac{p + \frac{z^2}{2n} \pm z\sqrt{\frac{p(1-p)}{n} + \frac{z^2}{4n^2}}}{1 + \frac{z^2}{n}} \tag{5}$$

5 Experimental Setup

5.1 AI models and prices

The study tests nine AI models, three from each of Anthropic, OpenAI, and Google. Table 3 lists their token prices from `config/prices.toml`. All models run with empty sampling parameters, because Anthropic endpoints reject `temperature`, `top_p`, and `top_k` since 21 October 2026. [**TODO:** State the run dates and the served model ids from the raw records.]

5.2 Budget and operational limits

The budget guard in `src/run/budget.py` enforces three limits in USD: $0.50 per request, $5.00 per model, and $10.00 per experiment. Before each Application Programming Interface (API) call, it bounds the request cost with Equation (6). Here L is the prompt length in characters and T is `max_output_tokens`.

$$C_{\max} = \frac{\lceil L/4 \rceil \, p_{\text{in}} + T \, p_{\text{out}}}{10^6} \le B_{\text{req}} \tag{6}$$

One pass sends 270 prompts per model, and each prompt has K attempts. Table 4 gives the worst-case cost per model for two settings. With $K = 3$ and $T = 2048$, the default model budget blocks six of the nine models, as Figure 4 shows.

A second estimate for the credit requests uses larger token counts. It assumes 810 attempts per model, 1,500 input tokens, and 4,096 output tokens on every attempt. This gives 109, 215, and 58 USD for the Anthropic, OpenAI, and Google models, which is about 380 USD in total. [**TODO:** Choose one budget basis for the paper and state the measured spend from the raw records.]

5.3 Statistical analysis

Each cell of model, setup, and validity label has a fixed number of attempts. With $K = 3$, a cell holds 180 attempts on valid inputs and 90 attempts on invalid

Table 3: AI models and token prices in United States Dollars (USD) per million tokens. Prices marked † rise to 4.00 and 20.00 on 1 December 2026. Prices marked ‡ rise to 1.50 and 7.50 on 1 January 2027.

AI model	Input	Output
`claude-opus-5-5`	4.00	20.00
`claude-sonnet-5-5`	2.00	10.00
`claude-haiku-5-5`	0.10	0.50
`gpt-6-astra`	10.00	50.00
`gpt-6.1-sol`	2.00	10.00
`gpt-6-luna`	0.10	0.50
`gemini-4-argon`†	2.00	10.00
`gemini-3.8-flash`‡	0.75	3.75
`gemini-3.5-flash-lite`	0.30	2.50

Table 4: Worst-case cost per model in USD for one pass of 270 prompts. The last column is the largest T that fits the default $5.00 model budget with $K = 1$.

AI model	K=1 T=1024	K=3 T=2048	max T at $5
`gpt-6-astra`	14.73	85.65	303
`claude-opus-5-5`	5.89	34.26	859
`claude-sonnet-5-5`	2.95	17.13	1784
`gpt-6.1-sol`	2.95	17.13	1784
`gemini-4-argon`	2.95	17.13	1784
`gemini-3.8-flash`	1.10	6.42	4871
`gemini-3.5-flash-lite`	0.72	4.23	7367
`claude-haiku-5-5`	0.15	0.86	36970
`gpt-6-luna`	0.15	0.86	36970

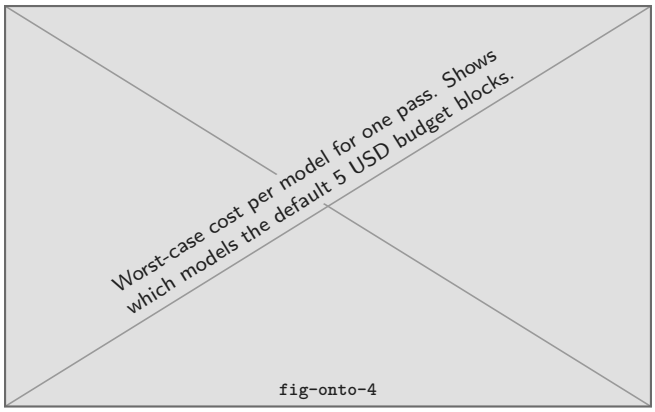


Figure 4: Worst-case cost per model for one pass of 270 prompts at $T = 1024$ and $K = 1$. The horizontal line marks the default model budget of $5.00. Hatched bars show the cost at $K = 3$ and $T = 2048$.

inputs. At these sizes, the 95% Wilson interval has a half width of at most 0.072 and 0.101. Differences smaller than these widths need a paired test across the shared inputs.

Uncertainty: paired comparison method

The repository computes Wilson intervals but no paired test between setups. The paper needs one test for the setup effect on the same inputs. [**TODO:** Choose McNemar, a paired bootstrap, or a mixed model, and state the correction for multiple comparisons.]

6 Results

Uncertainty: no measured results yet

No live LLM records exist in `results/` yet. Release 1.0.0 holds the placeholder prompt text of 18 October 2026, and Release 1.1.2 holds the approved text. All rates in this section stay unmeasured until the experiment runs on Release 1.1.2.

6.1 Hallucination rate by AI model and setup

Figure 5 shows the hallucination rate for each model and setup, split by valid and invalid inputs. [**TODO:** State the largest setup effect and the model that shows it.] The `prescriptive` setup changes the pooled rate from [**VALUE:** rate] to [**VALUE:** rate]. Figure 6 gives the paired change for each model, and Table 5 gives all rates with their intervals.

6.2 Outcome breakdown

A low hallucination rate can hide a high rate of malformed output or failure. Figure 7 therefore shows the share of all five outcomes for each model and setup. [**TODO:** Report whether rules in the prompt move attempts from hallucination to failure or to success.] The malformed output rate is [**VALUE:** range] across models.

6.3 Solvability detection

The mismatch flag turns solvability into a binary decision that a script can score. Figure 8 gives the confusion matrix for each setup, pooled over models. Figure 9 plots the mismatch detection rate against the false mismatch rate for each model. [**TODO:** State whether any setup lifts detection without a rise in false flags.]

Uncertainty: calibration of the flag

The reply contains a Boolean flag and no confidence value. The current records therefore do not support a calibration curve. A confidence field would change the output schema, which needs a new release and explicit approval of the project owner.

6.4 Difficulty scaling

Scramble depth sets an upper bound on the distance to solved. Figure 10 shows the success rate at depths 5, 10, and 20 for each model and setup. Figure 11 shows how output tokens grow with depth. [**TODO:** State whether rules in the prompt slow the fall of success with depth.]

6.5 Cost and accuracy

An ontology in the prompt costs input tokens on every attempt. It pays off only if it saves more than it adds. Figure 12 plots cost per success against success rate for each model and setup. [**TODO:** Name the models on the Pareto front.]

7 Pipeline Verification

Before any paid API call, we checked the pipeline against two test fixtures. The fixtures use a fake model and produce known outcomes on 2x2 inputs at depth 1. Table 6 shows that `evaluate` reproduces the expected rates and intervals.

8 Discussion

[**TODO:** Draft after results. Answer the question of the introduction in the first sentence.] The result must separate two effects of the rules in the prompt. One effect is fewer impossible moves, and the other is better detection of unreachable states. A setup can improve one effect and leave the other unchanged.

Strict string matching makes the hallucination count conservative. A model that writes a valid turn in an

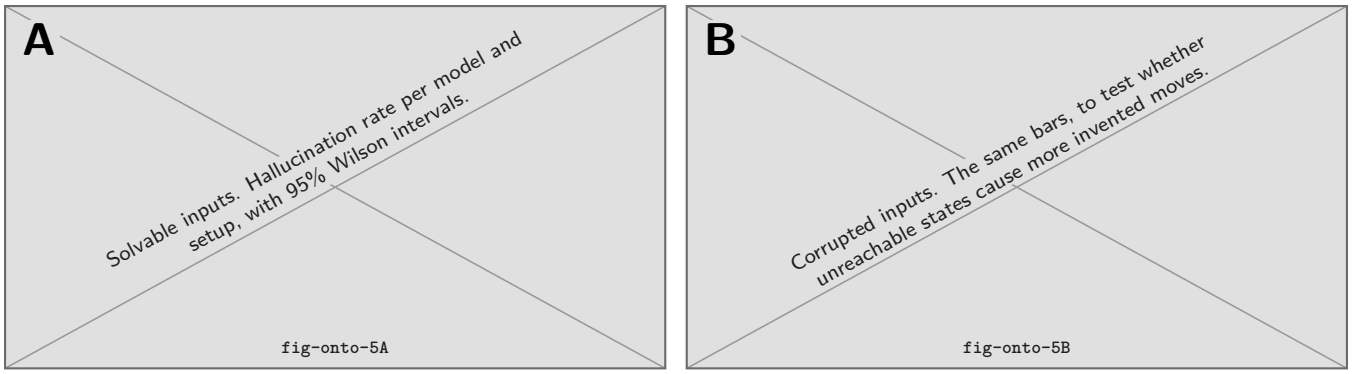


Figure 5: Hallucination rate by AI model and setup. Grouped bars give the share of attempts with at least one impossible move, with 95% Wilson intervals. (A) Valid inputs, $n =$ **[VALUE:** n**]** attempts per bar. (B) Invalid inputs, $n =$ **[VALUE:** n**]** attempts per bar. The bars sort the models by provider and price.

Table 5: Main results. Each cell gives the rate and its 95% Wilson interval for one AI model and setup. The columns cover success rate, hallucination rate, mismatch detection rate, false mismatch rate, and malformed output rate.

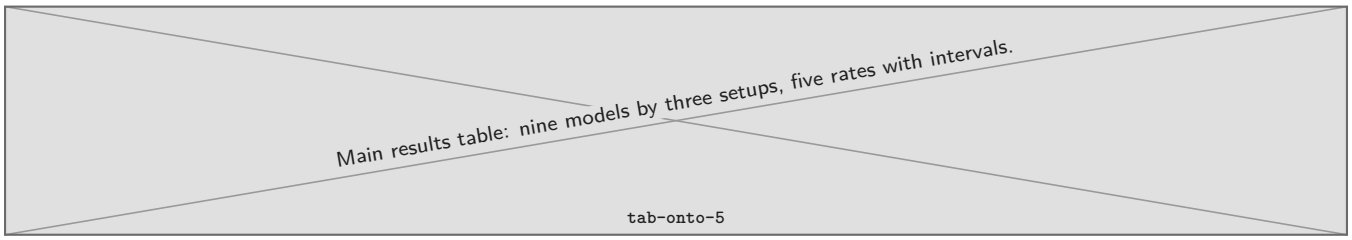


Table 6: Pipeline verification on fixture data. Each rate has its 95% Wilson interval. Detection is the mismatch detection rate. These numbers test the code path and say nothing about real AI models.

Fixture, model	Split	n per setup	Success	Detection	Malformed	E, C
`report_expected.md`, M	Valid	2	0.50 (0.09–0.91)	n/a; false 0.00	0.00 (0.00–0.66)	1.00, 0.50
`report_expected.md`, M	Invalid	2	0.50 (0.09–0.91)	0.50 (0.09–0.91)	0.50 (0.09–0.91)	n/a, 0.50
`pipeline-1.0.0.md`, A	Valid	4	0.00 (0.00–0.49)	n/a; false 0.00	0.00 (0.00–0.49)	n/a, 1.00
`pipeline-1.0.0.md`, A	Invalid	2	0.00 (0.00–0.66)	0.00 (0.00–0.66)	0.00 (0.00–0.66)	n/a, 1.00

unusual form, such as a prime sign in place of an apostrophe, gets a hallucination verdict. The appendix splits impossible moves by type, so a reader can see how many verdicts come from notation alone. **[TODO:** Report that share.]

The comparison rests on the assumption that the models already know Singmaster notation. If that assumption fails, the `no_model` setup is harder for reasons outside the ontology. The `descriptive` setup restates the notation and gives a partial control for this effect.

8.1 Limitations

Four engineering items limit the current scope of the benchmark. Each item is open in `AGENTS.md` and needs a decision of the project owner. The list below states the effect of each item on the results.

- The 3x3 reference solver is missing, and `reference_length()` raises an error for $n = 3$. Efficiency is therefore reported for the 2x2 cube only.
- Only the 2x2 and 3x3 cubes have engines. The octahedral and tetrahedral puzzles and the 5x5 cube have no engine yet.
- The validator matches move strings exactly and does not normalize them. Any format deviation in a move string counts as a hallucination.
- Providers bill hidden reasoning tokens in different ways, and API latency changes with load. Token and latency figures are therefore valid only for the run dates.

9 Reproducibility

Anyone can compute every number in the results again from the raw records, the release, and the configuration. The steps below rebuild the release, the reports, and this paper. Step 5 calls paid APIs, so it needs an approved budget.

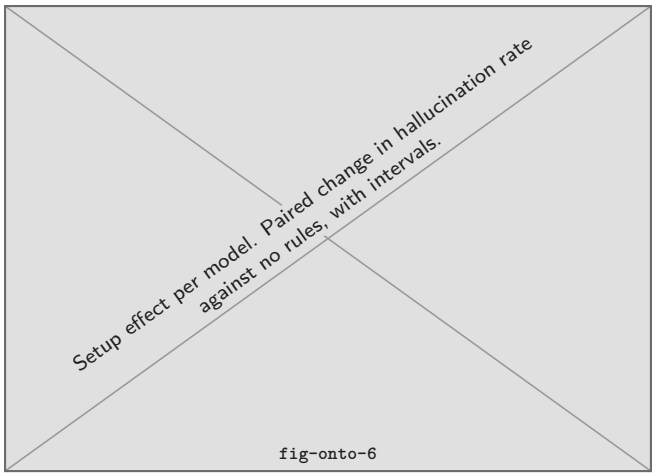


Figure 6: Change in hallucination rate for each AI model when the prompt adds rules. Circles show `descriptive` minus `no_model`, and squares show `prescriptive` minus `no_model`. Bars give 95% intervals from the paired test, [**TODO:** name the test].

1. Clone the repository and open a terminal in its root folder.
2. Run `pytest -q` and confirm zero failures.
3. Check out the tag `release-1.1.2`, which holds the frozen inputs and prompts.
4. Run the experiment with `--dry-run` to print the worst-case cost.
5. Run the experiment without `--dry-run` after you approve the budget.
6. Run `evaluate` to compute the reports from the raw records.
7. Run `make plots` to build every figure from the reports.
8. Build this paper with `latexmk -pdf main.tex`.
9. Run `python3 ste_check.py` and confirm that it reports zero violations.

Data and Code Availability

We will publish the frozen inputs, prompts, and raw records on Hugging Face with a Zenodo Digital Object Identifier (DOI). Code is under the Apache License 2.0, and data, prompts, and results are under CC BY 4.0. [**TODO:** Insert the dataset URL, the DOI, and the release tag.] [**TODO:** Confirm repository visibility, which is open in `AGENTS.md`.]

Acknowledgements

[**TODO:** Name the API credit programmes that fund the runs, if any award arrives.] The study applies to programmes from Anthropic, OpenAI, and Google. Each acknowledgement must match the terms of its award. We also thank [**TODO:** endorser and readers].

References

[1] [AUTHORS: TODO]. Cube Bench: A Benchmark for Spatial Visual Reasoning in MLLMs. arXiv:2512.20595, 2025. Code: github.com/dana-23/cube-bench.

[2] [AUTHORS: TODO]. The Illusion of Thinking. arXiv:2506.06941, 2025.

[3] [AUTHORS: TODO]. CubeBench: Diagnosing Interactive, Long-Horizon Spatial Reasoning Under Partial Observations. arXiv:2512.23328, 2025.

[4] [AUTHORS: TODO]. On the Limits of Innate Planning in Large Language Models. arXiv:2511.21591, 2025.

[5] [AUTHORS: TODO]. PlanBench: Evaluating Large Language Models on Planning and Reasoning about Change. arXiv:2206.10498, 2022.

[6] [AUTHORS: TODO]. Empirical Evidence of Complexity-Induced Limits in Large Language Models on Finite Discrete State-Space Problems with Explicit Validity Constraints. arXiv:2604.13371, 2026.

[7] [AUTHORS: TODO]. Can LLMs Follow Simple Rules? arXiv:2311.04235, 2023.

[8] [AUTHORS: TODO]. Evaluating Large Language Models with Grid-Based Game Competitions. arXiv:2407.07796, 2024.

[9] [AUTHORS: TODO]. LudoBench. arXiv:2604.05681, 2026.

[10] [AUTHORS: TODO]. HalluLens: LLM Hallucination Benchmark. arXiv:2504.17550, 2025.

[11] [AUTHORS: TODO]. Do Large Language Models Know What They Don't Know? arXiv:2305.18153, 2023.

[12] [AUTHORS: TODO]. Unsolvable Problem Detection: Evaluating Trustworthiness of Vision Language Models. arXiv:2403.20331, 2024.

[13] [AUTHORS: TODO]. From Blind Solvers to Logical Thinkers: Faulty Mathematical Problems. arXiv:2410.18921, 2024. Short title from docs/related-work.md; full title TODO.

[14] [AUTHORS: TODO]. Learning the Boundary of Solvability. arXiv:2512.01661, 2025.

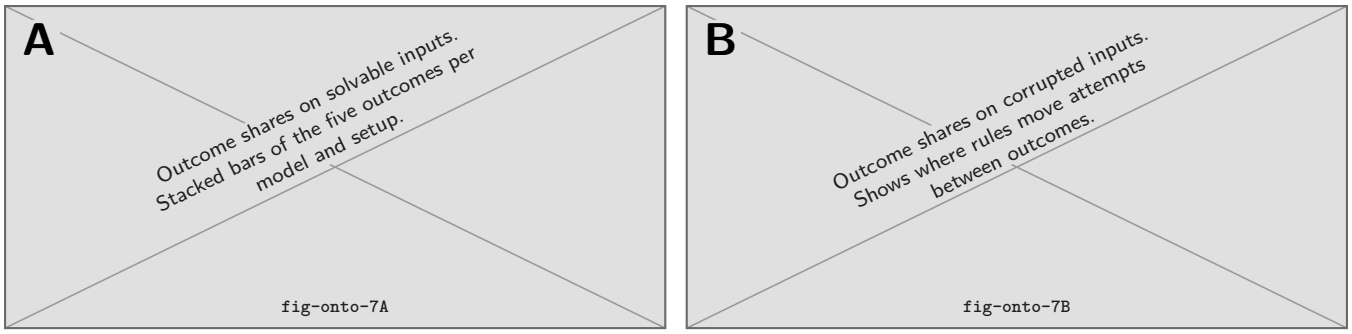


Figure 7: Outcome shares by AI model and setup. Each stacked bar splits the attempts into success, failure, hallucination, malformed output, and infrastructure failure. (A) Valid inputs. (B) Invalid inputs. Infrastructure failures stay out of every rate and appear here only as a count.

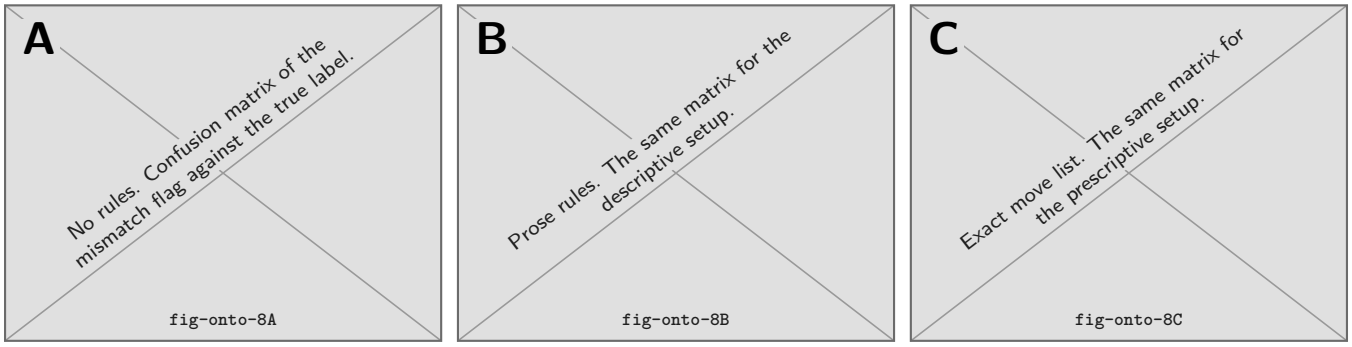


Figure 8: Solvability detection by setup, pooled over the nine AI models. Each two-by-two matrix counts flagged and unflagged attempts on valid and invalid inputs. (A) `no_model`. (B) `descriptive`. (C) `prescriptive`. Malformed output counts as unflagged. Per-model matrices are in the appendix.

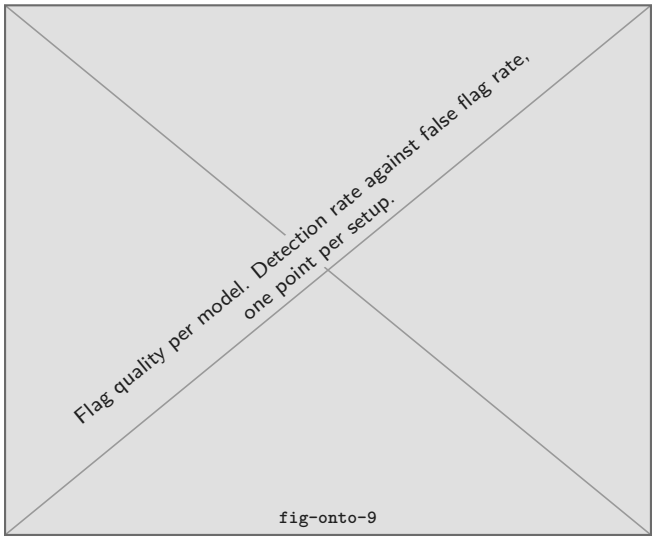


Figure 9: Quality of the mismatch flag. Each point is one AI model under one setup, with 95% Wilson intervals on both axes. Lines join the three setups of one model. The diagonal marks a flag that carries no information about reachability.

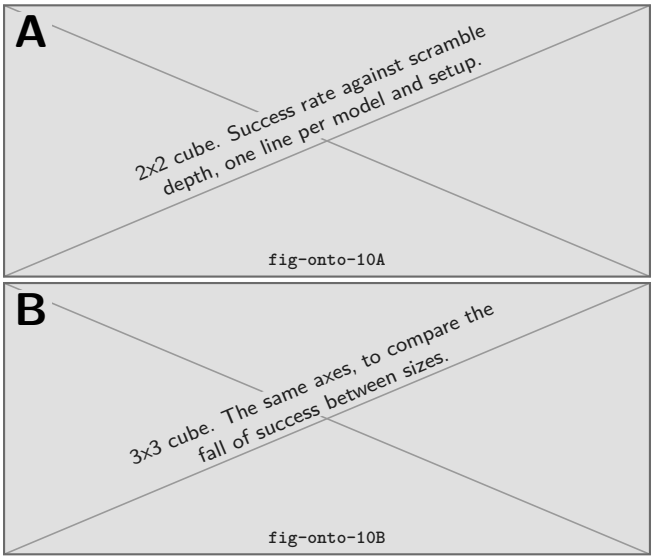


Figure 10: Success rate on valid inputs against scramble depth. (A) 2x2 cube. (B) 3x3 cube. Line style marks the setup and colour marks the model. Each point has $n =$ [**VALUE:** n] attempts and a 95% Wilson interval.

[15] Agrawal, Kumarage, Alghamdi, and Liu. Can Knowledge Graphs Reduce Hallucinations in LLMs? A Survey. In *Proceedings of NAACL 2024*, 2024. URL https://aclanthology.org/2024.naacl-long.219/. Surnames only in the sources; first names TODO.

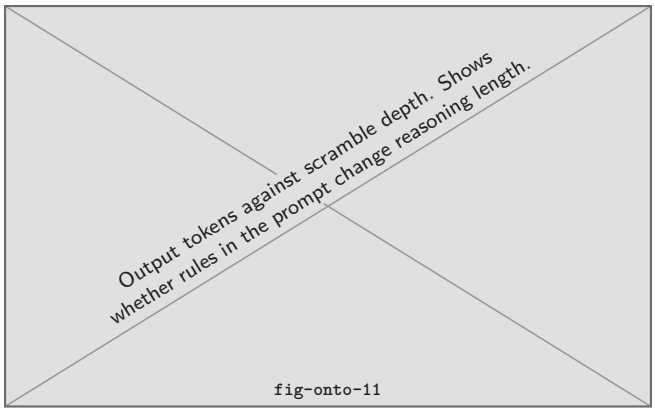


Figure 11: Mean output tokens per attempt against scramble depth, for each setup. Bands give the interquartile range over AI models. Output tokens include hidden reasoning tokens where the provider bills them.

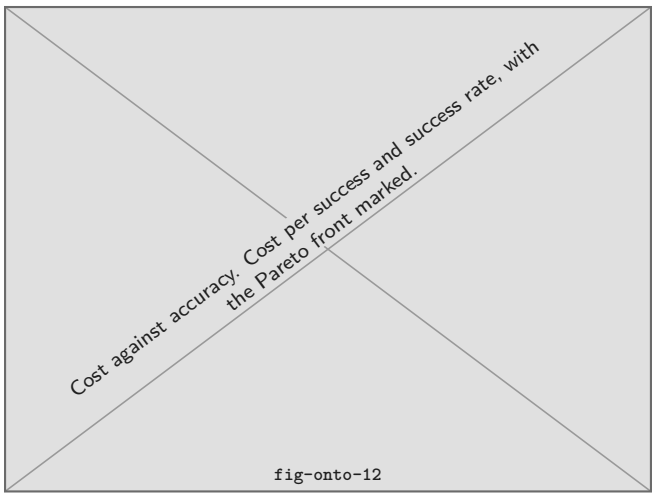


Figure 12: Cost per success in USD against success rate. Each point is one AI model under one setup. The line joins the points that no other point beats on both axes. The cost axis is logarithmic, and prices are those of Table 3.

[16] S. Penkov. Mitigating Hallucinations in Large Language Models via Semantic Enrichment of Prompts. In *Proceedings of CLIB 2024*, 2024. URL https://aclanthology.org/2024.clib-1.30/.

[17] Magaña Vsevolodovna and Monti. Enhancing Large Language Models through Neuro-Symbolic Integration and Ontological Reasoning. arXiv:2504.07640, 2025. Surnames only in the sources; first names TODO.

[18] Ali, Taha, and Morsey. Ontology-grounded Knowledge Graphs for Mitigating Hallucinations in Large Language Models for Clinical Question Answering. *Journal of Biomedical Informatics*, 175, 2026. doi: 10.1016/j.jbi.2026.104993. Full author names: TODO.

[19] Vlad Flaks and Mykhailo Flaks. ai-ontology-hallucination-research: source, frozen releases, and validator. GitHub repository vladflaks/ai-ontology-hallucination-research, 2026. Release 1.0.0, commit 3d7d8a6. Public URL and DOI: TODO (repository visibility is an open item).

[20] Edwin B. Wilson. Probable Inference, the Law of Succession, and Statistical Inference. *Journal of the American Statistical Association*, 22(158):209–212, 1927.

A Prompt Texts

The listings below give the approved 3x3 templates for the three setups. The runner fills `{state}` with the rendered state and `{legal_moves}` with the output of `legal_moves()`. The 2x2 templates differ only in the puzzle description.

Listing 1: `no_model`, 3x3.

```
Solve this 3x3 cube, written in Singmaster notation. The state is
      written as one block per face, headed by the face's letter.
      Each sticker is written as the letter of the face whose
      colour it has when the cube is solved. Rows are read top to
      bottom and stickers left to right, with each face seen from
      outside the cube: U with B at the top, D with F at the top,
      and R, F, L, and B with U at the top. Here is its current
      state:

{state}

If the state cannot be reached from the solved state by legal
     moves, set mismatch to true.
Reply with exactly this JSON object and nothing else: {"moves":
     [...], "mismatch": true|false}
List the moves in the order they are applied, one move per string.
```

Listing 2: `descriptive`, 3x3.

```
This is a 3x3 cube, written in Singmaster notation.

The puzzle: a cube with six faces, Up (U), Down (D), Right (R),
     Left (L), Front (F), and Back (B). Each face is a 3x3 grid
     of stickers. It has 8 corner pieces with three stickers each
     , 12 edge pieces with two stickers each, and 6 centres that
     never move relative to each other.

The state is written as one block per face, headed by the face's
     letter. Each sticker is written as the letter of the face
     whose colour it has when the cube is solved. Rows are read
     top to bottom and stickers left to right, with each face
     seen from outside the cube: U with B at the top, D with F at
      the top, and R, F, L, and B with U at the top.

Moves: a face letter alone, for example R, turns that face a
     quarter turn clockwise as seen looking at that face. A
     letter followed by an apostrophe, for example R', turns it a
      quarter turn anticlockwise. A letter followed by 2, for
     example R2, turns it a half turn. A turn moves the stickers
     of that face and the outer row of stickers on each of the
     four faces next to it.

The goal: a sequence of moves after which every face shows a
     single colour. The orientation of the whole cube does not
     matter.

Solve the cube. Here is its current state:

{state}

If the state cannot be reached from the solved state by legal
     moves, set mismatch to true.
```

```
Reply with exactly this JSON object and nothing else: {"moves":
     [...], "mismatch": true|false}
List the moves in the order they are applied, one move per string.
```

Listing 3: `prescriptive`, 3x3.

```
This is a 3x3 cube, written in Singmaster notation.

The puzzle: a cube with six faces, Up (U), Down (D), Right (R),
     Left (L), Front (F), and Back (B). Each face is a 3x3 grid
     of stickers. It has 8 corner pieces with three stickers each
     , 12 edge pieces with two stickers each, and 6 centres that
     never move relative to each other.

The state is written as one block per face, headed by the face's
     letter. Each sticker is written as the letter of the face
     whose colour it has when the cube is solved. Rows are read
     top to bottom and stickers left to right, with each face
     seen from outside the cube: U with B at the top, D with F at
      the top, and R, F, L, and B with U at the top.

Moves: a face letter alone, for example R, turns that face a
     quarter turn clockwise as seen looking at that face. A
     letter followed by an apostrophe, for example R', turns it a
      quarter turn anticlockwise. A letter followed by 2, for
     example R2, turns it a half turn. A turn moves the stickers
     of that face and the outer row of stickers on each of the
     four faces next to it.

The goal: a sequence of moves after which every face shows a
     single colour. The orientation of the whole cube does not
     matter.

Allowed moves: {legal_moves}

Every move you return must be one of the allowed moves, written
     exactly as listed.

Solve the cube. Here is its current state:

{state}

If the state cannot be reached from the solved state by legal
     moves, set mismatch to true.
Reply with exactly this JSON object and nothing else: {"moves":
     [...], "mismatch": true|false}
List the moves in the order they are applied, one move per string.
```

B Impossible Moves by Type

Strict matching counts every non-matching string as an impossible move. Figure 13 splits these strings into types, such as slice moves, wide moves, lowercase turns, whole-cube rotations, wrong prime signs, and invented tokens. The split is a post-hoc label and does not change any outcome. [**TODO:** Write the classifier and list its rules here.]

C Results by Puzzle

The main text pools the two cube sizes where the effect is the same. Figure 14 gives the success and hallucination rates for each size. [**TODO:** State whether the setup effect differs between the 2x2 and the 3x3 cube.]

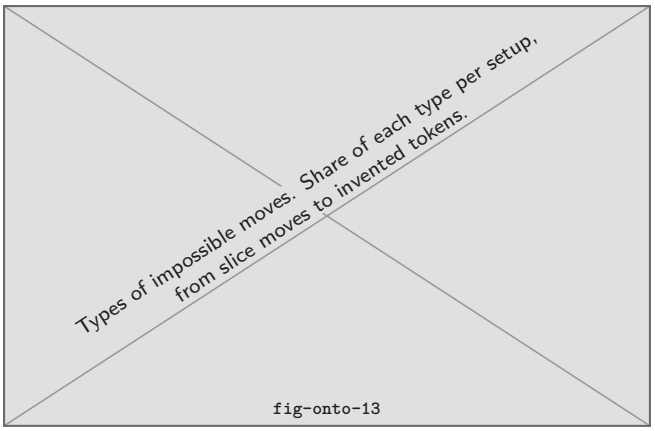


Figure 13: Impossible moves by type and setup, pooled over AI models. Each bar splits the impossible move strings into slice moves, wide moves, lowercase turns, rotations, wrong prime signs, and invented tokens. The counts are $n =$ [**VALUE:** n] strings per setup.

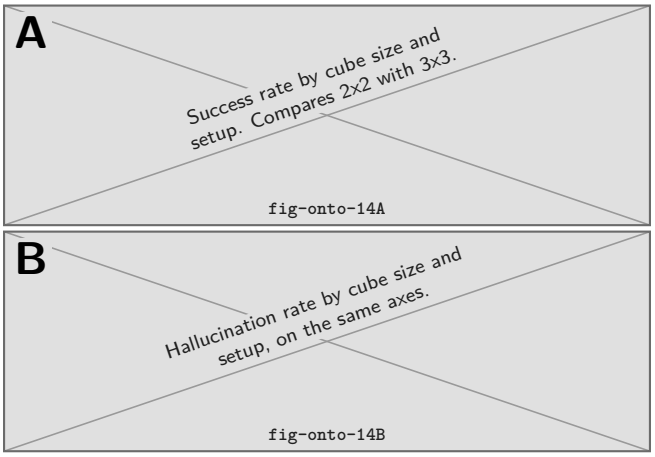


Figure 14: Results by puzzle. (A) Success rate and (B) hallucination rate for the 2x2 and 3x3 cubes under each setup, pooled over AI models, with 95% Wilson intervals. Future releases add the 5x5 cube and the octahedral and tetrahedral puzzles.

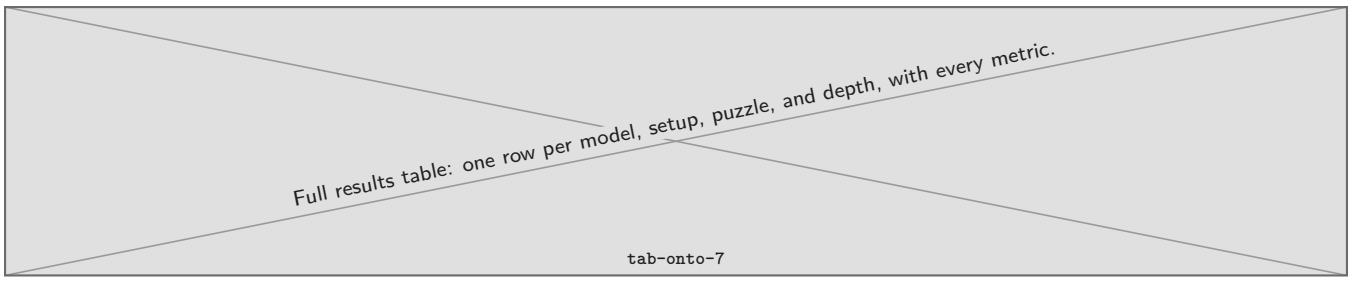
D Consistency and Efficiency

Each input gets K attempts for each model and setup. Figure 15 shows how often these attempts give the same outcome. Figure 16 compares the length of successful 2x2 solutions with the reference length.

E Full Results and Dataset Documentation

Table 7 gives every metric for each model, setup, puzzle, and depth. Table 8 lists the dataset fields that the NeurIPS Evaluations and Datasets Track requires. Both tables come from the same raw records as the main text.

Table 7: Full results for each AI model, setup, puzzle, and scramble depth. Columns give all rates with 95% Wilson intervals, plus efficiency, consistency, tokens per attempt, latency, and cost per success.



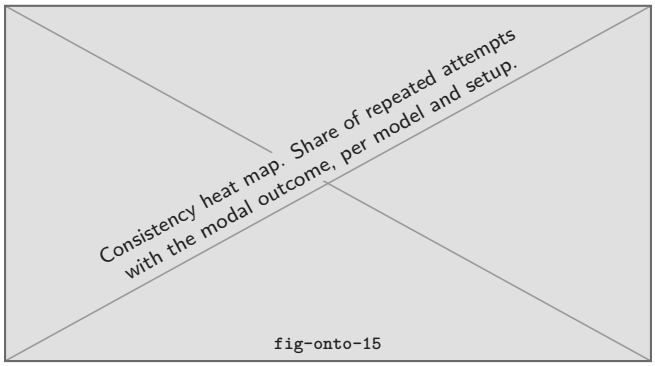


Figure 15: Consistency of outcomes over $K =$ **[VALUE:** K**]** repeated attempts. Each cell gives the mean share of the most common outcome for one AI model and setup. A value of 1.0 means that every attempt on an input gave the same outcome.

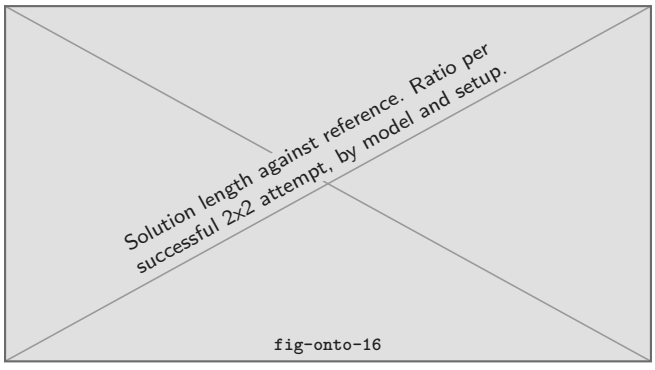


Figure 16: Efficiency of successful 2x2 solutions. Each box gives the ratio of move count to reference length for one AI model and setup. A ratio of 1.0 means a solution as short as the optimal 2x2 solver.

Table 8: Dataset documentation for the Croissant metadata file. The table lists the Responsible AI fields, the licence, the hosting location, and the DOI.

